

Small and simple things (Crypto)

1. Challenge Overview

We are given a script chal.py. Here are its contents:

```
flag = int(input("Enter flag: ").encode().hex(),16)

N =
0x355ba80ed8f6ed0e3d73f1e8dd90085db632fa60f031cd5b07dafda16d8fc0787af94
17481b7d52eb99b531d6d92e85be0bf2993e223c91e0a8676ae619847dba0f80e2777eb
c7e90a6dfd6ef0ef5f88a035c1d5ef56c2da799f2c6fb300343dc9c61d64908e7321b25
0a113fcab8dc777804f31926dfddbfc1ad22b54a3e64fe164bc3b890409c495de3af19
2a3838521ba6be9e57c71f0e9dcd14cacb454f133b9fa5548a47799305b47f7d54bb678
610cbe12fd7a0ef00e0f4918da6ed6b

c =
[0x3e568d71880ec0b281944ee7bb02eb2fddd128291d072b0f0b1b7fc6967f7ca95998
5ea033bf847018776364d4ffdc4b6610d0070563d60086de78e0532fb94d3b370a96808
0b7f87ac9d175edf5ce455d92068a0965185a679c8c995ccd1b37b58fa6476c555e100d
30310f8ef49e66a877747da4fd2cfc00f208741bafdf53b61392a1851af682099689b94
a5ec163dd840d81377bbe0db3c6de16deecdd629c19511a90bcfa3c51c08fdc92e6b32a
92d6b2291714416966512dffc1e9800a,

0x9aa3a3c39073307155aade05588c539e2079bcd0dae43092c4ae7479d3073c660f868
40e6f4128a766673d2dfa36f7d79575207e1835a5f68ea1856dd5f4d6965b814ca04b96
2db8c10c13908722176f6658b1a772156ebc490c8bc8b71c5e490e1666613fda97b6c60
dfa85b5a7c8c6a11cf3f37d9dbe9c18166067df22072ff64ca975092681906fc7ae3289
13648cdd8b6108d2608d58f45f7399b986a5a97266e6e81ed09ba6ce3f440fd3561d0ef
7e1d758149f608acd7962308ebc416a,

0x4c357d8139d9951ea48796e145cfa7a09199020cfabf734aaab35a02e8d88cac06234
9c29790ae30a72ee951d801c392781e9845f2a5d0a8e3e830a026f53b24845b0d272cea
249bdd1db72ef520ac5a363df374547e89425d351f394214b6514b4b9bee89279ffa9f1
c9d34483be31ae48f6c2f36d4d7a554fe9984f5f7f056faa78dda25c4e848f23a06556d
7d319e5e5d6db493ea09ce52ad5a27ac6f9a3b472e6d7e2f6380b1fb4f72cd4017f6911
4bd7c052b0ffc27de8af4f2dcecc8a6,

0x5f0cd34ee42c882d372cf1dab7e7049cb2f71a65d3be4c6107e45aaadf21be58e3e71
6b6fe9a42cb05e8ecd9574654539885bc95a57e2c9722231a04bb0a992d6c923a42cb9e
9ea7ccca78ef6ef36b42d1208b15843c43a8793fffd2758a391383ec9272d43acd3dfa3
2bdb989298e2301b7092807f580882810a174329e6045f5374772fce05f0c73ea85e67b
8a42bb0b541c59083d6c104c55d31d2d02cc2862b85d5a636fd11819827792a267fe223
9abb332be2ea917dd448476fdd6835b]
```

```

enc = sum(i*pow(flag,n,N) for n,i in enumerate(c))%N

if enc ==
0x34d45d13a2838e7e1835f747ca74355e4121f45347be4c4e589cb9e626c7be12fe100
76d97c78441ed32dbd99fec99d68bb62348d947831ff324a0404c87f730aa2e8fe9c325
9c84a10fabd721ae987e6821e24bc8ef079a904186ec8c6be5cbb6c8c83e5a348dc7287
17ff930c0eb9d4298b1b66a1cf6ffe253873cc19f859fe70af93c5395ba8779ffc8d38b
3fba9253d19c0dcc876e88b1bf8aaf0c9ffa000c381a3f875690029a0bef9c9013ead80
621158190a55b4cabfefc2bf0c20498:

    print("Congrats! You got the right flag")
else:

    print("Try again")

```

In the script, our flag is the unknown, we will represent it with the letter x.

The knowns are:

- N: This is given in hexadecimal format. As there are 512 characters, excluding '0x' at the beginning and each hexadecimal digit is 4 bits, N is a large integer of 2048 bits.
- c: This is a list of 4 large integers, which will become the coefficients of a polynomial equation.
- enc: This is the encrypted result of our flag, obtained after passing the flag into the polynomial function and wrapping the result around the modulus N.

Translating the python code into an equation $\text{sum}(i \cdot \text{pow}(\text{flag}, n, N) \text{ for } n, i \text{ in } \text{enumerate}(c)) \% N$, we get $\text{enc} = c_3 x^3 + c_2 x^2 + c_1 x + c_0 \pmod{N}$. The flag is the root of the equation.

Traditionally, the 2048-bit modulus N ensures security because solving for x in a modular ring is computationally equivalent to the Integer Factorization Problem. As the root evaluates to a value much larger than N, it wraps around the modulus multiple times. Without the prime factors of N, an attacker cannot determine how many wraps occurred, making it impossible to recover the flag using standard algebraic methods.

2. The Vulnerability

What then allows us to bypass the encryption? There are two main factors which contribute to the vulnerability in the encryption.

1. The degree of the polynomial is small. (3 in this case)
2. The root of the equation (flag) is approximately 50 characters, which is around 400 bits. This is significantly smaller than the 2048-bit modulus.

Recall the original equation, we are given that $enc = c_3x^3 + c_2x^2 + c_1x + c_0 \pmod{N}$. We can subtract enc from the constant c_0 to get

$f(x) = c_3x^3 + c_2x^2 + c_1x + c_0 - enc \equiv 0 \pmod{N}$. This is a univariate modular equation, where our flag, x , is the only unknown variable.

From here, we can obtain a solution in polynomial time using Coppersmith's method, as stated in Corollary 2.

Corollary 2. *Let $p(x)$ be a monic integer polynomial of degree k and N a positive integer of unknown factorization. In time polynomial in $\log N$ and k , we can find all integer solutions x_0 to $p(x_0) \equiv 0 \pmod{N}$ with $|x_0| < N^{1/k}$.*

Figure 1: Coppersmith's Theorem

Corollary 2 defines an upper bound for our root. Substituting our values, we have the following inequality: $2^{400} < 2^{\frac{2048}{3}}$. This specific configuration (small degree and root) violates the hardness assumptions that protect RSA-style modular equations. While finding roots of univariate polynomials over a ring of integers modulo N is generally difficult, we can now convert this into an integer space problem using Coppersmith's algorithm!

We will delve into the details of the algorithm in the next section, but for now, here is a short summary of how it works:

The Coppersmith algorithm creates a set of helper polynomials that all share the same root, which are then scaled by a bound X . We can then arrange the polynomials into a lattice basis and use LLL reduction to find a short vector. This shortest vector corresponds to a new polynomial that has such small coefficients it satisfies the Howgrave-Graham Lemma, which justifies why the modular root is also the integer root. Intuitively, if a number is too small to wrap around N , modular equality forces real equality.

Theorem 3. *Let $g(x)$ be an univariate polynomial with n monomials. Further, let m be a positive integer. Suppose that*

$$g(x_0) \equiv 0 \pmod{N^m} \quad \text{where} \quad |x_0| \leq X \quad (1)$$

$$\|g(xX)\| < \frac{N^m}{\sqrt{n}} \quad (2)$$

Then $g(x_0) = 0$ holds over the integers.

Figure 2: Howgrave-Graham's Theorem

3. Coppersmith's Method

Now that we have a background understanding of Coppersmith's algorithm, we are going to implement it with Sage. The following script is a modified version of David Wong's implementation:

1. Constructing $f(x)$ - When this polynomial evaluates to zero, the value of x is our recovered flag.

```
f = c[0] + c[1]*x + c[2]*x^2 + c[3]*x^3 - enc
```

2. Preprocessing - As Coppersmith's method requires the polynomial to be monic (leading coefficient has to be 1), we first multiply the entire polynomial by the modular inverse of the leading coefficient c_3 .

```
f_moniC = f * ZmodN(c[3])**-1
```

3. Define the search space - Note that this is denoted with XX in the code. As we had earlier estimated our flag size to be 400 bits, the search space will be 2^{400} . The search space can be anything less than the $N^{1/k}$, but if it is smaller than the flag size, the attack will fail.

```
XX = 2**400
```

4. Generate helper polynomials - In this step, we generate a collection of polynomials that all share the same root x_0 , but over a larger modulus N^m . (m is the multiplicity) These polynomials form the basis for our lattice. By using a larger modulus, we are able to generate more helper polynomials to provide the LLL algorithm with flexibility to find the optimal combination. x is also substituted with $x \cdot XX$ to create a weighted lattice that forces the LLL algorithm to prioritize minimizing the coefficients of higher-degree terms, which grow much faster.

```
gg = []

for ii in range(mm):

    for jj in range(dd):

        gg.append((x * XX)**jj * modulus**(mm - ii) * polZ(x *
XX)**ii)

    for ii in range(tt):

        gg.append((x * XX)**ii * polZ(x * XX)**mm)
```

5. Construct the Lattice Basis - The helper polynomials from the previous step are placed into a matrix BB . In this case, the nested loop only populates the i^{th} column of the i^{th} row, resulting in a triangular matrix. This guarantees full rank, meaning that

the helper polynomials are independent and provide unique information to the LLL (not redundant). It also makes it easier to calculate the determinant.

```
BB = Matrix(ZZ, nn)

for ii in range(nn):

    for jj in range(ii+1):

        BB[ii, jj] = gg[ii][jj]
```

6. LLL Reduction - The LLL algorithm combines the helper polynomials into a new polynomial with the smallest coefficients. The result is the shortest vector of the lattice. If it manages to find a vector small enough to satisfy the Howgrave-Graham bound, the attack has succeeded. Note that if LLL fails to find a root, the parameters XX, mm and tt can be adjusted.

```
BB = BB.LLL()
```

7. Solve for roots - Once the bound is satisfied, the modular equation can be solved over the integer space. Dividing by XX^{ii} undoes the scaling we had done in step 4.

```
new_pol = 0

for ii in range(nn):

    new_pol += x**ii * BB[0, ii] / XX**ii

potential_roots = new_pol.roots()
```

8. Verifying the roots - By substituting potential roots back into the original polynomial $f(x)$ and calculating the Greatest Common Divisor (GCD) with the modulus N, the script confirms whether the root satisfies the initial modular congruence $f(x) \equiv 0 \pmod{N}$.

```
9.     roots = []
10.    for root in potential_roots:
11.        if root[0].is_integer():
12.            result = polZ(ZZ(root[0]))
13.            if gcd(modulus, result) >= modulus**beta:
14.                roots.append(ZZ(root[0]))
15.    return roots
```

4. Final Solution

```
import time
import binascii
```

```

debug = True

# display matrix picture with 0 and X
def matrix_overview(BB, bound):
    for ii in range(BB.dimensions()[0]):
        row_str = f"{ii:02d} "
        for jj in range(BB.dimensions()[1]):
            row_str += '0' if BB[ii, jj] == 0 else 'X'
            row_str += ' '
        if BB[ii, ii] >= bound:
            row_str += '~'
        print(row_str)

def coppersmith_howgrave_univariate(pol, modulus, beta, mm, tt, XX):
    dd = pol.degree()
    nn = dd * mm + tt
    if not 0 < beta <= 1:
        raise ValueError("beta should belong in (0, 1]")
    if not pol.is_monic():
        raise ArithmeticError("Polynomial must be monic.")
    if debug:
        print("\n# Checking Bounds...")
        cond1 = RR(XX**(nn-1))
        cond2 = pow(modulus, beta*mm)
        print(f"*  $X^{(n-1)} < N^{(beta*m)}$ : {'GOOD' if cond1 < cond2 else 'NOT GOOD'}")

    detL = RR(modulus**(dd * mm * (mm + 1) / 2) * XX**(nn * (nn -
1) / 2))
    cond1 = RR(2**((nn - 1)/4) * detL**(1/nn))
    cond2 = RR(modulus**(beta*mm) / sqrt(nn))
    print(f"* LLL bound check: {'SOLUTION LIKELY' if cond1 < cond2
else 'UNLIKELY'}")

    # change ring of pol and x
    polZ = pol.change_ring(ZZ)
    x = polZ.parent().gen()

    # compute polynomials
    gg = []
    for ii in range(mm):
        for jj in range(dd):

```

```

        gg.append((x * XX)**jj * modulus**(mm - ii) * polZ(x *
XX)**ii)
    for ii in range(tt):
        gg.append((x * XX)**ii * polZ(x * XX)**mm)

    # construct lattice B
    BB = Matrix(ZZ, nn)
    for ii in range(nn):
        for jj in range(ii+1):
            BB[ii, jj] = gg[ii][jj]

    if debug:
        print("\n# Lattice Basis Overview:")
        matrix_overview(BB, modulus**mm)

    print("\nRunning LLL reduction (this may take a moment)...")
    BB = BB.LLL() # LLL

    # transform shortest vector in polynomial
    new_pol = 0
    for ii in range(nn):
        new_pol += x**ii * BB[0, ii] / XX**ii

    # factor polynomial
    potential_roots = new_pol.roots()

    # test roots
    roots = []
    for root in potential_roots:
        if root[0].is_integer():
            result = polZ(ZZ(root[0]))
            if gcd(modulus, result) >= modulus**beta:
                roots.append(ZZ(root[0]))
    return roots

# add challenge data
N =
0x355ba80ed8f6ed0e3d73f1e8dd90085db632fa60f031cd5b07dafda16d8fc0787af94
17481b7d52eb99b531d6d92e85be0bf2993e223c91e0a8676ae619847dba0f80e2777eb
c7e90a6dfd6ef0ef5f88a035c1d5ef56c2da799f2c6fb300343dc9c61d64908e7321b25
0a113fcab8dc777804f31926dfddbfc1ad22b54a3e64fe164bc3b890409c495de3af19
2a3838521ba6be9e57c71f0e9dcd14cacb454f133b9fa5548a47799305b47f7d54bb678
610cbe12fd7a0ef00e0f4918da6ed6b

```

```

c =
[0x3e568d71880ec0b281944ee7bb02eb2fddd128291d072b0f0b1b7fc6967f7ca95998
5ea033bf847018776364d4ffdc4b6610d0070563d60086de78e0532fb94d3b370a96808
0b7f87ac9d175edf5ce455d92068a0965185a679c8c995ccd1b37b58fa6476c555e100d
30310f8ef49e66a877747da4fd2cfc00f208741bafdf53b61392a1851af682099689b94
a5ec163dd840d81377bbe0db3c6de16deeccd629c19511a90bcfa3c51c08fdc92e6b32a
92d6b2291714416966512dffc1e9800a,

0x9aa3a3c39073307155aade05588c539e2079bcd0dae43092c4ae7479d3073c660f868
40e6f4128a766673d2dfa36f7d79575207e1835a5f68ea1856dd5f4d6965b814ca04b96
2db8c10c13908722176f6658b1a772156ebc490c8bc8b71c5e490e1666613fda97b6c60
dfa85b5a7c8c6a11cf3f37d9dbe9c18166067df22072ff64ca975092681906fc7ae3289
13648cdd8b6108d2608d58f45f7399b986a5a97266e6e81ed09ba6ce3f440fd3561d0ef
7e1d758149f608acd7962308ebc416a,

0x4c357d8139d9951ea48796e145cfa7a09199020cfabf734aaab35a02e8d88cac06234
9c29790ae30a72ee951d801c392781e9845f2a5d0a8e3e830a026f53b24845b0d272cea
249bdd1db72ef520ac5a363df374547e89425d351f394214b6514b4b9bee89279ffa9f1
c9d34483be31ae48f6c2f36d4d7a554fe9984f5f7f056faa78dda25c4e848f23a06556d
7d319e5e5d6db493ea09ce52ad5a27ac6f9a3b472e6d7e2f6380b1fb4f72cd4017f6911
4bd7c052b0ffc27de8af4f2dcecc8a6,

0x5f0cd34ee42c882d372cf1dab7e7049cb2f71a65d3be4c6107e45aaadf21be58e3e71
6b6fe9a42cb05e8ecd9574654539885bc95a57e2c9722231a04bb0a992d6c923a42cb9e
9ea7ccca78ef6ef36b42d1208b15843c43a8793fffd2758a391383ec9272d43acd3dfa3
2bdb989298e2301b7092807f580882810a174329e6045f5374772fce05f0c73ea85e67b
8a42bb0b541c59083d6c104c55d31d2d02cc2862b85d5a636fd11819827792a267fe223
9abb332be2ea917dd448476fdd6835b]

enc =
0x34d45d13a2838e7e1835f747ca74355e4121f45347be4c4e589cb9e626c7be12fe100
76d97c78441ed32dbd99fec99d68bb62348d947831ff324a0404c87f730aa2e8fe9c325
9c84a10fabd721ae987e6821e24bc8ef079a904186ec8c6be5cbb6c8c83e5a348dc7287
17ff930c0eb9d4298b1b66a1cf6ffe253873cc19f859fe70af93c5395ba8779ffc8d38b
3fba9253d19c0dcc876e88b1bf8aaf0c9ffa000c381a3f875690029a0bef9c9013ead80
621158190a55b4cabfefc2bf0c20498

# setup Polynomial Ring
ZmodN = Zmod(N)
P.<x> = PolynomialRing(ZmodN)

# build f(x) and make it monic
f = c[0] + c[1]*x + c[2]*x^2 + c[3]*x^3 - enc
f_mon = f * ZmodN(c[3])**-1

```

```

# attack parameters
beta = 1.0
XX = 2**400
mm = 4
tt = 1

print("Starting Coppersmith Attack...")
start_time = time.time()
roots = coppersmith_howgrave_univariate(f_monic, N, beta, mm, tt, XX)

if roots:
    print(f"\n{'='*30}")
    print("SUCCESS: ROOT FOUND")
    flag_int = int(roots[0])
    flag_hex = f"{flag_int:x}"
    if len(flag_hex) % 2 != 0:
        flag_hex = '0' + flag_hex
    print(f"Flag (int): {flag_int}")
    print(f"Flag (str): {binascii.unhexlify(flag_hex).decode()}")
    print(f"Time taken: {time.time() - start_time:.2f} seconds")
    print(f"\n{'='*30}")
else:
    print("\n[!] Failure: No root found. Try increasing mm/tt or XX.")

```

```

Starting Coppersmith Attack...

# Checking Bounds...
* X^(n-1) < N^(beta*m): GOOD
* LLL bound check: SOLUTION LIKELY

# Lattice Basis Overview:
00 X 0 0 0 0 0 0 0 0 0 0 0 0 ~
01 0 X 0 0 0 0 0 0 0 0 0 0 0 ~
02 0 0 X 0 0 0 0 0 0 0 0 0 0 ~
03 X X X X 0 0 0 0 0 0 0 0 0
04 0 X X X X 0 0 0 0 0 0 0 0 ~
05 0 0 X X X X 0 0 0 0 0 0 0 ~
06 X X X X X X X 0 0 0 0 0 0
07 0 X X X X X X X 0 0 0 0 0
08 0 0 X X X X X X 0 0 0 0 ~
09 X X X X X X X X X 0 0 0
10 0 X X X X X X X X X X 0 0
11 0 0 X X X X X X X X X 0
12 X X X X X X X X X X X X X

Running LLL reduction (this may take a moment)...

=====
SUCCESS: ROOT FOUND
Flag (int): 169731242419455455772251339823240894755509933250076678551981028894059288658020584301802490942446836338150630525
Flag (str): HEX{sagemath_is_a_wonderful_and_magical_world}
Time taken: 0.23 seconds
=====

```

Figure 3: Sage Cell output

Parameters:

- $XX = 2^{400}$

This is the root bound. Set it to the estimated flag size and increase if the attack fails.

- $mm = 4$

This is the multiplicity. A higher mm makes the math easier but the script slower. Increase if the script's LLL bound check says UNLIKELY.

- $tt = 1$

This is the number of extra shifts. Adding more shifts increases the lattice dimension and improves the chances of finding a shorter vector, but it also increases runtime.

Also, the lattice dimensions (nn) are given by $dd \times mm + tt$.

5. Flag Recovery

After running the LLL reduction, the shortest vector provided a polynomial whose integer root was:

- **Integer Root:**
1697312424194554557722513398232408947555099332500766785519810288940
59288658020584301802490942446836338150630525
- **Decoded Flag:** HEX{sagemath_is_a_wonderful_and_magical_world}

6. Learning Resources

- Survey: Lattice Reduction Attacks on RSA by David Wong (2015):
https://www.davidwong.fr/papers/david_wong_rsa_lll_boneh_durfee__2015.pdf
- Finding a Small Root of a Univariate Modular Equation by Don Coppersmith (1996):
https://static.aminer.org/pdf/PDF/000/192/854/finding_a_small_root_of_a_univariate_modular_equation.pdf
- Finding small roots of univariate modular equations revisited by Nicholas Howgrave-Graham (1997): <https://link.springer.com/chapter/10.1007/bfb0024458>